Texas A&M University

ECEN689 Project Report for Group 7

Implementation and Analysis of PEPITA: A Forward Learning Algorithm

Authors:

Gudivada Venkata Prudvi Tej - 436000703

Yash Kamble - 136004988

Yashas Bharadwaj - 635009332

Date: November 26, 2024

1 Motivation

The backpropagation algorithm, despite its success in training artificial neural networks, is fundamentally at odds with the way biological neural systems operate. Real neurons learn through local updates and feedback driven by temporal dynamics, not global error signals or synchronized weight updates. Forward learning algorithms aim to mimic these biologically inspired processes, offering a more natural and adaptive approach to training artificial networks. One such algorithm is discussed in detail in this report.

2 Introduction

The **PEPITA algorithm** (*Present the Error to Perturb the Input To modulate the Activity*) is a **forward-learning approach** that uses top-down feedback connections, aiming to address the limitations of **backpropagation** in biological plausibility [2]. Instead, PEPITA sends *feedback* signals from later layers (higher up in the hierarchy) back to earlier layers, helping them make corrections during learning.

3 Algorithm

i. Clean Pass

In a standard feedforward pass (called the *clean pass*), the network computes the activations at each layer. The input x is passed through the layers to produce hidden activations h_ℓ, where ℓ is the layer index. The activations follow the equation:

$$h_1 = \sigma_1(W_1 x), \quad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}) \quad \text{for } 2 \leq \ell \leq L$$

Here:

- W_ℓ is the weight matrix for layer ℓ.
- σ_ℓ represents the non-linear activation function at each layer.
- L is the final layer, where the output h_L is generated.

ii. Modulated Pass

After the clean pass, the paper introduces a *modulated pass,* which adds an error modulation component to the input layer. In this pass, the input is adjusted by projecting the network's output error back to the input layer using a random fixed matrix F. The modulated input is:

$$x - Fe$$

where:

- y is the one-hot encoded target output.
- $e = h_L - y$ is the error between the network's final layer output h_L and the target y.

The activation computations during the modulated pass are updated to reflect this change:

$$h_1^{\text{err}} = \sigma_1 \left(W_1(x - Fe) \right), \quad h_\ell^{\text{err}} = \sigma_\ell \left(W_\ell h_{\ell-1}^{\text{err}} \right) \quad \text{for } 2 \leq \ell \leq L$$

These modulated activations (h_ℓ^{err}) represent the altered response of each layer when fed an input that has been modified by the feedback error.

iii. Weight Updation

After the two forward passes (clean and modulated), PEPITA adjusts the weights based on the differences between the clean and modulated activations at each layer. This rule encourages the network to reduce the error indirectly by adjusting weights to close the gap between clean and modulated activations. Specifically:

For the first layer:

$$\Delta W_1 = (h_1 - h_1^{\text{err}})(x - Fe)^T$$

This change is based on the difference between clean and modulated activations, scaled by the modulated input $x - Fe$.

For intermediate layers $2 \leq \ell < L$:

$$\Delta W_\ell = (h_\ell - h_\ell^{\text{err}})(h_{\ell-1}^{\text{err}})^T$$

The weights adjust based on the discrepancy between clean and modulated activations, with each layer responding to the error from the layer above it.

For the last layer:

$$\Delta W_L = e(h_{L-1}^{\text{err}})^T$$

Here, the weights are adjusted directly by the error e and the modulated activation of the preceding layer.

4 Implementation

The implementation of the PEPITA algorithm was carried out using MATLAB. The complete source code for the implementation is available at the following link: **MATLAB Implementation of PEPITA Algorithm**.

The algorithm was implemented and evaluated using the MNIST dataset [1], a widely used benchmark for image classification tasks. The MNIST dataset consists of 60,000 training images of handwritten digits (0-9). The goal is to classify the images into one of the 10 possible classes. The dataset was preprocessed by normalizing the pixel values to the range [0, 1] and flattening the images into 1D vectors.

Hyperparameters & Network Specifications

Several hyperparameters were tuned during the implementation to evaluate the performance of the algorithm under different conditions. Following are the values when we obtain our best results:

- **Input Size:** The network has 64 input neurons, corresponding to the 8x8 pixel images of the MNIST dataset. This means each image is flattened into a 64-dimensional vector.
- **Hidden Layer Size:** The network has 50 neurons in the hidden layer.
- **Output Size:** The output layer consists of 10 neurons, each corresponding to one of the digits (0-9).
- **Batch Size:** The batch size was set to 100, meaning that the network processes 100 training samples at once during each forward and backward pass.
- **Number of Batches:** A total of 600 batches were used, corresponding to the number of batches needed for the entire training dataset.
- **Learning Rate (η):** The learning rate was set to 0.5, which controls the magnitude of weight updates during training.
- **Number of Epochs:** The network was trained for 3 epochs.

Activation Function

The sigmoid activation function was used in the hidden layer to introduce non-linearity, defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

5 Results

The performance of the PEPITA algorithm was evaluated over the course of training using accuracy as the metric. The following plot shows the accuracy of the model over all iterations, which is the product of the number of batches and epochs. The plot visualizes how the model's accuracy improved as it learned from the training data.

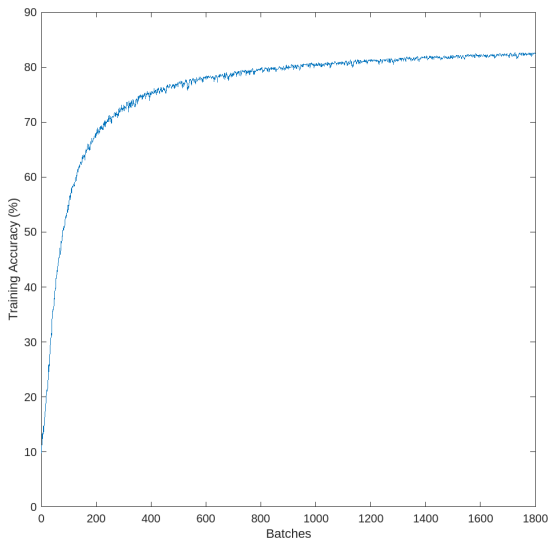

Figure 1: Accuracy of the PEPITA algorithm over all iterations (batch number times epochs)

5.1 Visualizing the Gradients

In this section, we compare the accuracy of calculating the gradient of our implementation of PEPITA with the weight perturbation method of estimating gradients.

Weight Perturbation: For a neural network with weights W, where each weight is represented by w_i, the gradient is approximated using the weight perturbation method. The gradient for a weight w_i is calculated as:

$$\frac{\partial E}{\partial w_i} \approx \frac{E(w + \delta w_i) - E(w)}{\delta w_i}$$

where δW_i is the small perturbation to the weight and E represents the loss function.

5.1.1 Input Image 1

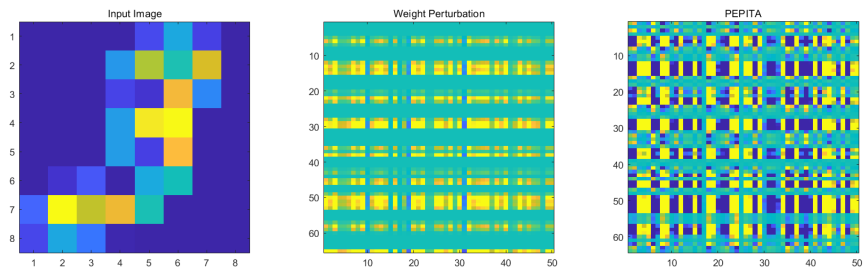

Figure 2: Comparison of gradients for w_1 using weight perturbation and PEPITA

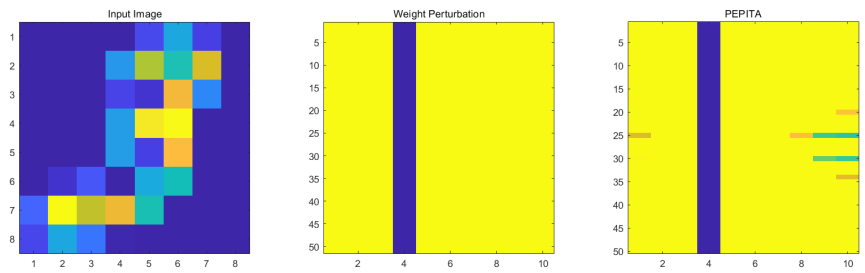

Figure 3: Comparison of gradients for w_2 using weight perturbation and PEPITA

5.1.2 Input Image 2

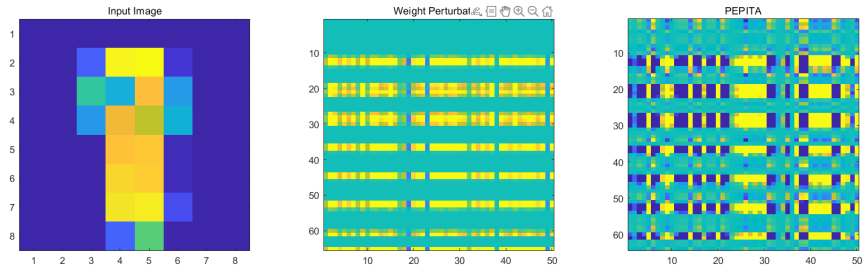

Figure 4: Comparison of gradients for w_1 using weight perturbation and PEPITA

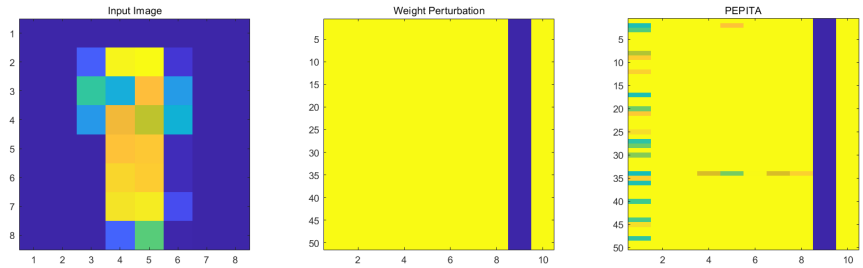

Figure 5: Comparison of gradients for w_2 using weight perturbation and PEPITA

6 Adding Noise

To account for potential inaccuracies in weight updates caused by noise, we model the noise in the weight adjustments using a Gaussian distribution. Specifically, we generate random noise for the weight matrix W using the MATLAB command:

```
noise_W = normrnd(0, 0.1, size(W));
```

where the noise is drawn from a normal distribution with a *mean of 0* and *a standard deviation of 0.1*. This noise is then added to the weight updates at the end of each batch to simulate the impact of stochastic variations on the learning process.

The plot visualizes the model's accuracy as it learns from the training data with the addition of noise. We can clearly observe that the model performs worse compared to the previous model, which had no noise added to it. The noise negatively impacts the weight updates, leading to reduced accuracy over time.

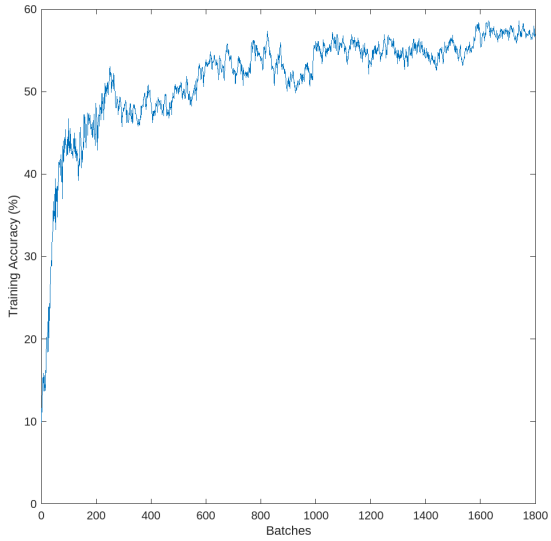

Figure 6: Accuracy of the PEPITA algorithm over all iterations with noise added

7 Inference

The accuracy results show a significant drop when noise is added, with the model achieving about 82% accuracy without noise but struggling to reach 60% with noise, highlighting the impact of noise on training; additional epochs may be required for the noisy model to approach the same level of accuracy.

References

[1] Yann LeCun, Corinna Cortes, and Christopher J.C. Burges. *MNIST handwritten digit database*. http://yann.lecun.com/exdb/mnist/. 2010.

[2] Ravi Francesco Srinivasan et al. "Forward Learning with Top-Down Feedback: Empirical and Analytical Characterization". In: *ICLR*. 2024. URL: https://openreview.net/forum?id=My7lkRNnL9.